

Premessa:

Riassunto fatto utilizzando anche chat gpt, scritto per il ripasso pre-orale. Quindi omessi qualsiasi tipo di esercizio.

Introduzione al Calcolatore e Architettura di Sistema

Il panorama tecnologico attuale offre una vasta gamma di strumenti di calcolo, ognuno progettato per scopi specifici e con caratteristiche tecniche differenti. Di seguito viene presentata una classificazione dettagliata dei calcolatori e un'analisi dei componenti fondamentali che ne permettono il funzionamento secondo il modello di riferimento classico.

1. Tipologie di Calcolatori

Esistono diverse categorie di calcolatori, classificati in base al loro utilizzo, alla loro potenza e alla loro architettura:

- **Calcolatori Embedded:** Si tratta di sistemi **integrati all'interno di una macchina più grande**. Il loro scopo principale è quello di **controllare e monitorare un processo specifico**. Generalmente, questi dispositivi non sono progettati per essere riprogrammati dall'utente finale.
- **Calcolatori Personali (PC):** Questa categoria include i comuni **desktop** e i **laptop**. Sono caratterizzati da un'estrema versatilità e vengono impiegati per una **grande varietà di applicazioni**, come lo studio, il lavoro, l'intrattenimento, la navigazione web e la creazione di documenti.
- **Workstation:** All'interno dei calcolatori personali, le workstation rappresentano una sottocategoria specifica. Sono **calcolatori molto più performanti** rispetto ai modelli standard e vengono utilizzati prevalentemente in contesti lavorativi che richiedono elevata potenza di calcolo.
- **Server e Sistemi Enterprise:** Sono sistemi significativamente **più potenti** dei calcolatori personali. Il loro compito è quello di gestire una potenza di calcolo superiore per **servire le richieste degli utenti** trasmesse attraverso la rete e offrire servizi specifici.
- **Supercalcolatori e Calcolatori Grid:** Questi sistemi sono progettati per offrire **prestazioni ed elevate potenze di calcolo**. Sebbene garantiscano capacità di elaborazione massime, comportano costi di realizzazione e mantenimento molto elevati.
- **Calcolatori Cloud:** Sono infrastrutture di calcolo che **forniscono servizi direttamente attraverso Internet**.

2. Il Modello di Von Neumann

Le componenti fondamentali che costituiscono un calcolatore moderno derivano da quello che viene definito **Modello di Von Neumann**. Questo schema architettonico identifica alcuni elementi chiave necessari per l'elaborazione dei dati:

1. **Rete di Interconnessione (Bus):** Il canale che permette lo scambio di informazioni tra le varie componenti.

2. Memoria Centrale e Memoria Secondaria: Dedicate alla conservazione dei dati e delle istruzioni.

3. Periferiche di Input e di Output: Interfacce per l'interazione con l'esterno.

4. Processore (CPU): Composto dall'**Unità Aritmetico-Logica (ALU)** e dall'**Unità di Controllo**.

2.1 Differenza tra Istruzioni e Dati

All'interno di un programma, è fondamentale distinguere tra due tipi di informazioni:

- **Istruzioni:** Rappresentano i **comandi del programma** e definiscono il compito specifico che deve essere svolto. Un istruzione è data da codice operativo (indica l'azione che compierà l'istruzione) e operandi (dati).

- **Dati:** Sono i valori (numeri o caratteri) che vengono utilizzati dalle istruzioni come **operandi** per poter eseguire i compiti previsti.

2.2 Parola di memoria

I processori di oggi, infatti, sentiamo molto spesso dire che riescono a gestire dati fino a 32 o 64 bit.

La parola di memoria rappresenta la quantità di bit che il processore può elaborare contemporaneamente come unità base. Nelle architetture moderne questa dimensione coincide generalmente con quella dei registri, pari a 32 o 64 bit.

parola di memoria

3. Le Unità di Ingresso e Uscita (I/O)

Le periferiche di ingresso e uscita rappresentano il punto di contatto tra il calcolatore e l'utente o l'ambiente esterno.

- **Unità di Ingresso (Input):** Servono per **immettere informazioni nel calcolatore**. Esempi tipici sono la tastiera, il mouse, il microfono e la videocamera.

- **Unità di Uscita (Output):** Sono i componenti che **forniscono i dati elaborati dal calcolatore all'utente**. Tra questi troviamo lo schermo, le stampanti e le cuffie (senza microfono).

- **Dispositivi Ibridi:** Esistono dispositivi che svolgono contemporaneamente entrambe le funzioni.

- La **stampante con scanner integrato** agisce come output (stampa) e input (scansione).

- Le **cuffie con microfono** combinano l'input del microfono e l'output degli auricolari.

- Il **touchscreen** (come quello degli smartphone) funge da output mostrandoci le immagini, ma permette anche l'input di informazioni tramite il tocco delle dita.

4. L'Unità di Memoria

La memoria si suddivide in due categorie principali con funzioni e caratteristiche differenti: la memoria centrale e la memoria di massa (secondaria).

4.1 Memoria Centrale

La memoria centrale è composta principalmente da:

- **RAM (Random Access Memory):** È una memoria **veloce ma volatile**, il che significa che necessita di alimentazione elettrica costante per mantenere le informazioni. Realizzata con transistor e semiconduttori, la RAM ha il compito di contenere i **dati e le istruzioni dei programmi in fase di esecuzione**.
- **Memoria Cache:** Si posiziona fisicamente e logicamente tra la RAM e il processore. È una memoria **estremamente veloce**, ma rispetto alla RAM è più costosa e ha una capacità ridotta (più piccola).

4.2 Memoria RAM Statica vs memoria RAM Dinamica

- **RAM dinamica:** questo tipo di RAM nonostante ci sia l'alimentazione tende a perdere i dati memorizzati se non rinfrescati, poiché col passare del tempo la corrente all'interno della cella di memoria viene dissipata. Il refresh nella RAM dinamica consiste nella rigenerazione periodica della carica elettrica delle celle di memoria, poiché questa tende a dissiparsi nel tempo, anche in presenza di alimentazione.
- **RAM statica:** a differenza della precedente questo tipo di memoria non ha bisogno di rinfrescare il dato, per cui il processo di lettura di quest'ultimo avviene molto più velocemente alla memoria RAM dinamica, però è ovviamente molto più costosa in termini di quantità di transistor (**ES: Cache**)

4.3 Memoria Secondaria (Memoria di Massa)

A differenza della RAM, la memoria secondaria è **persistente**: i dati rimangono memorizzati anche in assenza di alimentazione elettrica. È caratterizzata da un **costo inferiore** rispetto alla memoria centrale.

5. Il Processore (CPU)

Il processore è il "cuore" del calcolatore e, secondo lo schema semplificato del modello di Von Neumann, contiene due elementi essenziali:

- **ALU (Arithmetic Logic Unit):** È l'unità di elaborazione che esegue materialmente i **calcoli matematici e le operazioni logiche**.
- **Unità di Controllo (CU):** Ha il compito di **coordinare i vari stadi del processore** durante l'esecuzione di un'istruzione. Essa invia segnali specifici basati sullo stato del processore e si occupa di **scandire il tempo dei flussi di informazione**.

*Nota: Sebbene il modello di Von Neumann sia la base, architetture più avanzate come quelle **CISC** e **RISC** introducono ulteriori elementi di dettaglio nel funzionamento dei processori.*

Introduzione alle Periferiche e al Modello di Von Neumann

All'interno del **modello di von Neumann**, le **operazioni di ingresso e di uscita (I/O)** e le relative periferiche giocano un ruolo essenziale. La loro presenza è fondamentale per evitare che i programmi siano puramente deterministici; grazie alle periferiche, infatti, il risultato di un programma può dipendere dall'interazione con l'utente e dagli input forniti, rendendo l'esito non scontato.

Per permettere al processore di accedere alle periferiche, esistono sostanzialmente due metodologie:

- **Unificazione degli indirizzi di memoria:** a ogni periferica viene associato un indirizzo di memoria specifico attraverso il quale viene identificata.
- **Istruzioni specifiche:** vengono utilizzate istruzioni dedicate appositamente alla gestione e all'identificazione delle periferiche.

Struttura delle Periferiche e Registri

Una componente cruciale di ogni periferica è costituita dai suoi **registri interni**, che non devono essere confusi con i registri del processore. Questi si dividono principalmente in tre categorie: **registri di dati (data)**, **registri di stato (status)** e **registri di controllo**.

Una premessa necessaria riguarda la velocità di funzionamento: il processore e le periferiche operano a **velocità molto differenti**. Di conseguenza, è indispensabile un meccanismo di **sincronizzazione** che permetta una comunicazione efficace tra le due entità.

Il caso dell'Input: la Tastiera

Prendendo come esempio la tastiera, quando viene premuto un tasto, un circuito di controllo codifica l'input in formato binario utilizzando lo standard **ASCII**. In questo contesto intervengono i registri specifici:

- **KBD data:** un registro a 8 bit che contiene il carattere immesso.
- **KBD status:** contiene un bit fondamentale chiamato **K-in**. Se il bit K-in è impostato a **1**, indica che la tastiera è pronta a ricevere o ha un nuovo dato disponibile; se è a **0**, indica che il dato è in fase di lettura dal registro KBD data.

Il caso dell'Output: il Display

Il funzionamento delle periferiche di output, come il display, è speculare a quello di input. Anche qui troviamo:

- **DISP data:** un registro a 8 bit che ospita il carattere da visualizzare.
- **DISP status:** contiene il bit **D-out**. Se D-out è a **1**, la periferica è pronta per visualizzare un nuovo carattere; se è a **0**, il sistema sta scrivendo sul registro DISP data.

Le Interruzioni (Interrupts)

Le interruzioni sono un elemento cardine nella comunicazione tra periferica e processore. Quando una periferica necessita dell'attenzione del processore, invia un segnale di richiesta chiamato **IRQ**

(Interrupt Request). Se la richiesta può essere soddisfatta, il processore sospende l'esecuzione del programma corrente per saltare alla **routine di servizio dell'interruzione (ISR)**.

Gestione della Routine di Servizio

La routine di servizio è paragonabile alla chiamata di un sottoprogramma. Tuttavia, poiché il **Program Counter (PC)** deve essere aggiornato per puntare alla routine, è necessario salvare il valore attuale del PC (tramite un registro o una pila) per non perdere il punto di rientro nel programma principale.

Al termine della routine, viene eseguita l'istruzione **Return from Interrupt (RFI)**, che ripristina il PC all'indirizzo successivo all'istruzione interrotta. Inoltre, il processore conferma alla periferica la ricezione della richiesta tramite un segnale di **IACK (Interrupt Acknowledgement)**.

Le routine di servizio delle interruzioni fanno parte del software di sistema (sistema operativo o firmware) e non sono scritte dal programmatore applicativo.

Salvataggio dello Stato del Processore

Prima di servire un'interruzione, è vitale salvare lo **stato del processore** per evitare la perdita di dati durante la routine. Le informazioni da salvare includono:

- I **registri** di lavoro.
- Il **Program Counter (PC)**.
- I bit di esito o di stato, che formano il **Program Status (PS)**.

Per preservare il registro PS, si può utilizzare una copia in un registro ausiliario o un banco di registri dedicato.

Abilitazione e Priorità delle Interruzioni

In certe situazioni, può essere utile che il processore non serva interruzioni, ad esempio per evitare l'**annidamento delle interruzioni** (ovvero un'interruzione che ne interrompe un'altra).

Il Bit IE (Interrupt Enable)

Per gestire questa possibilità si utilizza il **bit IE**:

- **Nel processore:** abilita o disabilita la capacità globale di servire richieste di interruzione.
- **Nella periferica:** abilita o disabilita la possibilità della singola periferica di inviare una richiesta.

Livelli di Priorità

Un altro metodo per gestire l'annidamento è l'uso dei **livelli di priorità**. Se una periferica con priorità più alta invia una richiesta mentre il processore ne sta servendo una a priorità più bassa, quest'ultima viene interrotta per servire la nuova richiesta. In caso contrario, il processore prosegue con la routine corrente. Per evitare conflitti simultanei tra più periferiche, esiste un **circuito di arbitraggio** che gestisce le precedenze a monte.

Ciclo Completo di un'Interruzione: Esempio della Tastiera

Il processo completo segue questa sequenza logica:

1. L'utente preme un tasto; il circuito lo codifica in **ASCII**.
 2. Il bit **K-in** nel registro di stato viene posto a **1**, segnalando la presenza del dato nel registro data.
 3. La periferica invia un segnale di **IRQ**.
 4. Il processore sospende il programma, salvando **PC** e **PS**.
 5. Il bit **IE** viene posto a **0** per evitare annidamenti immediati.
 6. Il processore invia un segnale di **IACK** alla periferica e avvia la routine di servizio.
 7. Terminata la routine, vengono ripristinati **PC**, **PS** e il bit **IE** viene riportato a **1**.
-

Gestione di Periferiche Multiple

Quando più periferiche comunicano con il calcolatore, il processore deve identificare chi ha inviato la richiesta e quale routine eseguire. Esistono due tecniche principali:

- **Polling**: il processore scansiona una per una le periferiche verificando il bit IRQ. Segue un ordine di priorità prestabilito, ma questa tecnica diventa inefficiente all'aumentare del numero di periferiche.
 - **Interruzioni Vettorizzate**: la periferica invia, insieme alla richiesta, un codice di pochi bit che funge da indice per una **tabella degli indirizzi delle interruzioni**. In questo modo, il processore riconosce immediatamente la periferica e la routine associata.
-

Registri di Controllo del Processore e Eccezioni

I principali registri coinvolti nella gestione del sistema sono:

- **PS (Program Status)**: contiene i bit di stato e il bit IE.
- **IPS**: registro ausiliario usato per copiare il PS quando inizia una routine.
- **IE (Enable)**: registro che abilita le richieste delle periferiche.
- **IP Pending**: registro che indica il numero di interruzioni attive o in attesa di essere servite.

Eccezioni vs Interruzioni

Un'eccezione è un evento sincrono perché nasce dall'istruzione che il processore sta eseguendo in quel momento, quindi è legata al flusso del programma.

Un'interruzione invece è asincrona perché arriva dall'esterno, ad esempio da una periferica, e può verificarsi in qualsiasi istante indipendentemente dall'istruzione corrente.

Le interruzioni generate dalle periferiche sono eventi asincroni perché, pur essendo abilitate e gestite dal programma tramite istruzioni di I/O, il momento in cui la periferica segnala l'evento non è determinato da una specifica istruzione del flusso di esecuzione, ma da un evento esterno. Anche se il programma utilizza istruzioni di I/O per configurare o leggere una periferica, l'interruzione è asincrona perché viene generata dalla periferica in risposta a un evento esterno e può arrivare in qualunque punto dell'esecuzione, indipendentemente dall'istruzione corrente.

Esse includono:

Tipi eccezioni

- **Eccezioni di memoria:** tentativi di leggere da indirizzi inesistenti, alterati o protetti.
- **Errori di calcolo:** come la divisione per zero.

Se si verifica un errore critico di questo tipo, il sistema solitamente decide di interrompere l'esecuzione del programma principale

. Componenti Hardware del Processore

La struttura fondamentale di un processore è composta da diversi elementi hardware coordinati tra loro per l'esecuzione delle operazioni. I componenti principali includono:

- **Banco dei registri (Register File):** un'area di memoria interna veloce utilizzata per memorizzare temporaneamente i dati durante l'elaborazione.
- **ALU (Arithmetic Logic Unit):** l'unità preposta all'esecuzione dei calcoli logici e matematici.
- **Generatore di indirizzi e Program Counter (PC):** circuiti dedicati alla gestione dell'ordine di esecuzione delle istruzioni. Il PC punta all'indirizzo dell'istruzione da eseguire.
- **Registro IR (Instruction Register):** contiene l'istruzione attualmente in fase di esecuzione dopo che è stata prelevata dalla memoria.
- **Circuiti di controllo:** coordinano il flusso dei dati tra le varie unità del processore.
- **Interfaccia Processore-Memoria:** gestisce lo scambio di informazioni (dati e istruzioni) tra la CPU e la memoria esterna.

2. L'Esecuzione delle Istruzioni: La Struttura a 5 Stadi

Nell'architettura **RISC** (e parzialmente in quella **CISC**), l'esecuzione di un'istruzione viene suddivisa in **cinque stadi sequenziali**. Idealmente, ogni stadio corrisponde a un **ciclo di clock**.

1. Prelievo (Fetch): tramite il registro **PC**, si preleva l'istruzione dalla memoria e la si carica nel registro **IR**.

2. **Decodifica e Lettura Registri:** l'istruzione in IR viene interpretata dai circuiti di controllo e vengono letti i valori necessari dal banco dei registri.

3. **Elaborazione (Execute):** l'**ALU** compie i calcoli logici, matematici o il calcolo degli indirizzi di memoria.

4. **Accesso alla Memoria (Memory):** avviene la lettura o la scrittura (Load/Store) di dati nelle locazioni di memoria.

5. **Scrittura nei Registri (Write Back):** il risultato finale dell'operazione viene scritto nel registro di destinazione all'interno del banco dei registri.

3. Analisi Dettagliata delle Istruzioni tramite Esempi

Per comprendere come questi stadi interagiscano con l'hardware, analizziamo tre tipologie fondamentali di istruzioni.

Istruzione di caricamento: **LOAD R5, X(R7)**

Questa istruzione legge un valore dalla memoria e lo salva in un registro. Il suo scopo è sommare un valore immediato **X** al contenuto del registro **R7** per trovare l'indirizzo di memoria da cui leggere il dato da scrivere in **R5**.

- **Stadio 1 (Fetch):** Prelievo dell'istruzione tramite PC e caricamento in IR.
- **Stadio 2 (Decode/Read):** Decodifica dell'istruzione e lettura del registro **R7**, il cui valore viene salvato nel registro intermedio **RA**.
- **Stadio 3 (Execute):** L'ALU calcola l'indirizzo effettivo sommando il valore di **RA** (contenente R7) e il valore immediato **X** (selezionato dal multiplexer **MuxB**). Il risultato viene salvato nel registro intermedio **RZ**.
- **Stadio 4 (Memory):** Si legge il dato dalla locazione di memoria all'indirizzo contenuto in **RZ**. Il dato letto viene posto nel registro intermedio **RY**.
- **Stadio 5 (Write Back):** Il valore in **RY** viene infine scritto nel registro di destinazione **R5**.

Istruzione aritmetica: **ADD R3, R4, R5**

Questa istruzione somma il contenuto di R4 e R5, salvando il risultato in R3.

- **Stadi 1 e 2:** Prelievo e lettura dei registri **R4** e **R5**, che vengono caricati rispettivamente in **RA** e **RB**.
- **Stadio 3:** L'ALU effettua la somma tra i valori in RA e RB, depositando il risultato in **RZ**.
- **Stadio 4:** Non è richiesto accesso alla memoria. Tuttavia, il dato passa attraverso il multiplexer **MuxY**, che seleziona il risultato grezzo dell'ALU da RZ.
- **Stadio 5:** Il risultato viene scritto nel registro di destinazione **R3**.

Istruzione di memorizzazione: **STORE R6, X(R8)**

Questa istruzione salva il contenuto del registro R6 nella memoria, all'indirizzo calcolato sommando X a R8.

- **Stadio 1 e 2:** Prelievo e lettura di **R6** e **R8**. R8 viene caricato in **RA**, mentre R6 viene caricato in **RB** e successivamente nel registro intermedio **RM** per essere preservato senza passare per l'ALU.
 - **Stadio 3:** L'ALU calcola l'indirizzo sommando **RA** (R8) e il valore immediato **X**. Il risultato va in **RZ**.
 - **Stadio 4:** Scrittura del valore contenuto in **RM** (R6) all'indirizzo di memoria specificato da **RZ**.
 - **Stadio 5:** Viene saltato, poiché non è prevista alcuna scrittura nel banco dei registri.
-

4. Caratteristiche del Datapath nei Processori RISC

Il **datapath** è l'insieme dei componenti che gestiscono il passaggio dei dati durante l'esecuzione.

Registri Intermedi (Registri di Stadio)

Tra uno stadio e l'altro sono presenti registri intermedi come **RA, RB, RZ, RM, RY**. Essi hanno il compito di **conservare l'output di uno stadio** affinché possa essere utilizzato come input per lo stadio successivo, garantendo la fluidità del flusso dei dati.

Gestione del Banco dei Registri

Il banco dei registri dispone di porte specifiche:

- **Porte A e B:** utilizzate per l'uscita dei dati (lettura).
- **Porta C:** utilizzata per l'ingresso dei dati (scrittura).

I Multiplexer (Mux)

I multiplexer agiscono come selettori per decidere quale dato deve proseguire nel datapath:

- **MuxB:** decide se inviare all'ingresso B dell'ALU un valore letto dai registri (RB) oppure un **valore immediato** presente nell'istruzione.
 - **MuxY:** posto dopo lo stadio di esecuzione, seleziona se inviare allo stadio di Write Back il risultato dell'ALU (da RZ) o il dato letto dalla memoria (da RY).
 - **MuxMA:** gestisce l'accesso alla memoria. Se il selettore è a **1**, si preleva un'istruzione (input dal PC); se è a **0**, si accede a un **dato** (input dal registro RZ).
-

5. Gestione del Flusso di Esecuzione (PC e Salti)

Il processore deve sapere costantemente quale istruzione eseguire successivamente.

- **Incremento Sequenziale:** In un'architettura RISC, ogni istruzione occupa una **parola di memoria** da 32 bit (4 byte). Per passare all'istruzione successiva, il **Program Counter (PC)** viene incrementato di **4**.

- **Gestione dei Salti (Branch):** In caso di salti, il PC non viene incrementato di 4, ma di un valore maggiore calcolato sommando al PC un valore immediato (offset).
- **MuxINC e MuxPC:**
 - **MuxINC:** se impostato a 0, somma 4 al PC; se impostato a 1 (salto), somma al PC un valore immediato.
 - **MuxPC:** seleziona la sorgente per il nuovo valore del PC. Se impostato a 1, accetta l'uscita del sommatore dedicato; se a 0, accetta un valore dal registro **RA** (per salti a indirizzi specifici contenuti in un registro).

6. Formato delle Istruzioni e Decodifica

Le istruzioni RISC hanno una dimensione fissa di **32 bit**. La suddivisione dei bit varia in base al tipo di istruzione:

- **Registri:** Ogni registro richiede **5 bit** per essere identificato (permettendo di indirizzare fino a 32 registri).
- **Istruzioni Registro-Registro (es. ADD):** 3 registri occupano 15 bit (5x3), i restanti sono per il **codice operativo (opcode)**.
- **Istruzioni con Valore Immediato (es. ADDI):** 2 registri (10 bit), un valore immediato (16 bit) e l'opcode (5 bit).
- **Load/Store:** Simili alle immediate, ma i 16 bit del valore immediato sono usati come **spostamento (displacement)**.
- **Salti Condizionati (es. BGT):** 2 registri (10 bit), un'etichetta di salto (nello spazio dell'operando immediato) e l'opcode.
- **Salti Incondizionati:** Opcode (5 bit) e il resto dello spazio dedicato al valore immediato del salto.

Nota sull'efficienza: Nello Stadio 2, il processore è in grado di decodificare l'istruzione e leggere i registri contemporaneamente perché la posizione dei bit dei registri all'interno dell'istruzione è fissa. L'hardware legge sempre i registri in quelle posizioni; se l'istruzione non li richiede, quei valori verranno semplicemente ignorati.

L'Esecuzione a Stadi e il Segnale MFC

Nel modello hardware del processore organizzato a stadi, l'esecuzione di un'istruzione avviene seguendo un numero **n** di fasi. In un contesto ideale, ogni stadio dovrebbe corrispondere esattamente a un **ciclo di clock**, tuttavia esistono delle eccezioni pratiche legate principalmente alle operazioni che richiedono un **accesso alla memoria**. È infatti possibile completare un accesso alla memoria in un singolo ciclo di clock solo se il dato o l'istruzione necessari si trovano già nella memoria cache; in caso contrario, il tempo richiesto sarà superiore.

Per gestire questa asincronia, viene impiegato un segnale specifico denominato **MFC (Memory Function Completed)**. Questo segnale viene impostato a "true" (vero) solo quando l'operazione di lettura o scrittura è stata effettivamente portata a termine. L'MFC risulta fondamentale durante il prelievo delle istruzioni (fase di fetch) e durante lo scambio di dati con la memoria, rendendo i segnali di controllo per il **datapath** essenziali, in particolare nei passi 1 e 4 dell'esecuzione.

Gestione dei Segnali di Controllo e dei Registri

I **segnali di controllo** coordinano il funzionamento dell'intero processore agendo su diversi componenti:

- **Multiplexer:** I selettori che determinano quale ingresso deve essere scelto in un multiplexer sono gestiti direttamente dai segnali di controllo.
- **Abilitazione dei Registri:** I segnali permettono di attivare o disattivare la scrittura nei registri. I **registri intermedi** (quelli posti tra uno stadio e l'altro) rimangono **sempre abilitati**, poiché i dati scritti in uno stadio servono immediatamente come input per lo stadio successivo. Al contrario, registri come il **PC (Program Counter)** o l'**IR (Instruction Register)** vengono disabilitati e la loro scrittura è autorizzata solo negli stadi specifici in cui il loro contenuto deve essere effettivamente modificato.
- **ALU e Memoria:** Esistono segnali come **ALUop**, che indica la tipologia di operazione che l'unità aritmetico-logica deve eseguire, e i segnali **MemLettura** e **MemScrittura**, che comandano le operazioni verso la memoria. Anche l'abilitazione alla scrittura in registri è gestito da segnali come **RF Scrittura**.

Fondamenti dell'Architettura dei Processori: Connessioni, Funzionamento e Prestazioni

1. La Connessione tra Processore e Memoria

Il funzionamento di un sistema informatico si basa sulla stretta **connessione fra il processore e la memoria**. All'interno del processore risiedono alcuni registri di indirizzamento che risultano essenziali per la **fase di prelevamento delle istruzioni** (fetch) dalla memoria. I due registri fondamentali per questo processo sono il **PC (Program Counter)** e l'**IR (Instruction Register)**.

- **PC (Program Counter):** È un puntatore che indica l'**istruzione successiva** da eseguire, ovvero punta alla prossima istruzione nel flusso del programma.
- **IR (Instruction Register):** Questo registro contiene l'**istruzione corrente**, cioè l'istruzione che si sta effettivamente eseguendo in quel momento.

Il caricamento dell'IR avviene proprio tramite il PC. In termini operativi, viene interrogata la memoria attraverso un **accesso diretto**; si recupera l'istruzione puntata dal PC e la si scrive all'interno dell'IR. Questo scambio è reso possibile da un'apposita **interfaccia processore-memoria** situata all'interno dell'unità centrale, che gestisce l'accesso attraverso specifici **segnali di controllo di scrittura e lettura**.

2. Gestione delle Interruzioni e Interazione con l'Utente

Durante l'esecuzione di un'istruzione, può accadere che il processore riceva un **segnale di interruzione**. In questo caso, il processore deve sospendere l'attività corrente per entrare in una specifica **routine di interruzione**.

Una volta terminata tale routine, è fondamentale che venga **ripristinato lo stato dell'elaborazione precedente** all'interruzione. Questo passaggio garantisce che l'esecuzione possa riprendere esattamente dal punto in cui era stata interrotta senza perdite di dati o errori di flusso. Le interruzioni rivestono un ruolo vitale per diverse ragioni:

- Permettono ai programmi di **sfruttare le periferiche**.
- Consentono la **comunicazione con l'utente**.
- Rendono i programmi **non deterministici**, permettendo loro di essere influenzati dall'**input dell'utente** in tempo reale.

3. Le Prestazioni del Processore e la Tecnologia dei Transistor

Le **prestazioni di un processore** derivano principalmente dalla sua **velocità** e dalla **tecnologia** costruttiva impiegata. Per comprendere la velocità, è essenziale analizzare il ruolo dei **transistor**, che costituiscono i circuiti logici di base.

I transistor hanno il compito di **decodificare la corrente elettrica** che li attraversa in valori binari, ovvero **0 e 1**. La velocità di esecuzione di un'istruzione è direttamente collegata alla rapidità con cui la corrente passa attraverso questi circuiti: una maggiore velocità di transito permette una trasmissione dei dati superiore.

È importante notare che la velocità è **inversamente proporzionale alle dimensioni dei transistor**. Di conseguenza:

- Più i transistor sono **piccoli**, più è possibile utilizzarne in numero elevato all'interno del chip.
- Dimensioni ridotte consentono una maggiore velocità nelle operazioni di **elaborazione, lettura e scrittura**.

4. Organizzazione dell'Hardware e Parallelismo

Un altro pilastro che determina le prestazioni complessive è l'**organizzazione dell'hardware**, che oggi si fonda principalmente sul concetto di **parallelismo**. Il parallelismo può essere implementato a diversi livelli:

- **Livello di istruzioni:** Si realizza attraverso l'uso della **pipeline**, che permette di ottimizzare la sequenza di esecuzione dei comandi.
- **Livello di core:** Prevede l'adozione di un'**architettura multicore**. In questo schema, ogni core rappresenta un'unità di elaborazione distinta del processore, dotata di una propria **cache dedicata**.
- **Livello di sistema:** Fa riferimento a configurazioni composte da **più processori** che comunicano tra loro, i quali possono anche condividere porzioni di memoria per collaborare alla risoluzione dei compiti.

1. L'Instruction Set Architecture (ISA)

L'insieme di tutte le istruzioni che un processore è in grado di riconoscere e interpretare viene definito **ISA (Instruction Set Architecture)**. L'ISA rappresenta un elemento fondamentale nel calcolatore poiché **funge da ponte tra l'hardware e il software**, permettendo ai programmatori di scrivere codice che sia effettivamente eseguibile dal processore.

Nello specifico, l'ISA definisce:

- Il nome e la tipologia delle **istruzioni** utilizzabili.
- I **codici operativi** e gli **operandi** (ovvero i dati) che possono essere combinati.
- Le modalità con cui i dati possono essere manipolati.

Un programma è essenzialmente un insieme di queste istruzioni che, nel caso base (ovvero quando non si utilizza una pipeline), viene **eseguito in modo sequenziale** dal processore. Inoltre, l'esecuzione di un programma permette l'interazione con l'utente attraverso le **interfacce di input e output**, a seconda di come il software è stato progettato.

2. Organizzazione della Memoria e Indirizzamento

Dal Linguaggio di Alto Livello al Binario

Mentre linguaggi come il C richiedono un **compilatore** per tradurre il codice di alto livello in linguaggio assembler, per quest'ultimo non è necessario un compilatore, bensì un **assemblatore**. Il compito dell'assemblatore è prendere il programma scritto in assembly e tradurlo in **sequenze binarie** comprensibili dalla macchina.

Struttura della Memoria

Le informazioni all'interno della memoria sono organizzate sotto forma di un **vettore di parole**, ovvero una successione di parole di memoria. Ogni parola ha una lunghezza variabile, generalmente compresa tra **16 e 64 bit**. A ogni parola del vettore è associato un **indirizzo di memoria binario e univoco**, che permette di accedere alla specifica allocazione per prelevare dati o istruzioni.

La quantità totale di parole che la memoria può contenere è definita **spazio di indirizzamento**. Ad esempio, se avessimo un indirizzamento basato su 24 bit, lo spazio di indirizzamento totale sarebbe pari a 2²⁴ parole di memoria.

Unità di Misura e Dati

I dati e le istruzioni sono sequenze di cifre binarie. Per quantificare l'informazione si utilizzano le seguenti equivalenze:

- **1 Byte** = 8 bit.
- **1 Kilobyte (KB)** = 2¹⁰ byte.
- **1 Megabyte (MB)** = 2²⁰ byte.
- **1 Gigabyte (GB)** = 2³⁰ byte.
- **1 Terabyte (TB)** = 2⁴⁰ byte.

Schemi di Indirizzamento: Big-Endian e Little-Endian

L'indirizzamento avviene assegnando un indice (indirizzo) ai byte contenuti in ciascuna parola. Esistono due tipologie di organizzazione:

- Nel formato Big-Endian il byte più significativo di una parola è memorizzato all'indirizzo di memoria più basso.
- Nel formato Little-Endian il byte meno significativo è memorizzato all'indirizzo di memoria più basso.

3. Operazioni di Memoria e Notazione RTN

Le operazioni principali effettuate sulla memoria sono:

- **Load:** Caricamento di un dato letto dalla memoria in un registro.
- **Store:** Salvataggio di un dato in una specifica locazione di memoria.
- **Fetch:** Fase essenziale in cui si preleva un'istruzione dalla memoria per essere eseguita.

Per descrivere queste operazioni e il trasferimento dei dati si utilizza la **notazione RTN (Register Transfer Notation)**. Questa notazione usa simboli specifici:

- I registri del processore sono identificati come **R1, R2, R3**, ecc..
- Le **parentesi quadre** vengono utilizzate per indicare l'operazione di spiazzamento o l'accesso al contenuto.
- Vengono usate etichette per identificare **variabili e costanti**.

4. Architetture RISC e CISC

Nel linguaggio assembly, le istruzioni si dividono in due grandi famiglie a seconda dell'architettura del processore:

• RISC (Reduced Instruction Set Computer):

- Le istruzioni hanno limiti precisi: hanno lunghezza fissa di **una parola di memoria** (es. 32 bit).
- Il numero di operandi è limitato.
- A causa della semplicità delle istruzioni, il programmatore deve spesso utilizzarne un numero maggiore per completare un'operazione complessa.

• CISC (Complex Instruction Set Computer):

- Non presenta i limiti di lunghezza della RISC; un'istruzione può essere più grande di una parola di memoria.
- Può gestire un **numero maggiore di operandi**.
- Le istruzioni sono intrinsecamente più complesse e potenti.

5. Modi di Indirizzamento

I modi di indirizzamento definiscono come il processore individua l'operando su cui deve lavorare. Si distinguono i seguenti metodi:

1. **Modo Registro:** Il dato è contenuto direttamente all'interno di un **registro** del processore.
2. **Modo Assoluto:** Il dato è contenuto in una **variabile** o in una costante identificata da un'etichetta.
3. **Modo Immediato:** Il valore del dato è indicato direttamente nell'istruzione, solitamente preceduto dal simbolo **cancelletto (#)**.
4. **Modo Indiretto:** Utilizza un registro come **puntatore**; il registro non contiene il dato, ma l'**indirizzo di memoria** dove il dato è effettivamente memorizzato.
5. **Modo con Indice e Spiazzamento:** L'indirizzo finale si ottiene sommando il contenuto di un registro (che funge da indice) a un valore di spiazzamento (variabile o costante).
 - *Esempio:* L'istruzione `Load R2, X(R5)` indica che l'indirizzo da cui leggere il dato è dato dalla somma tra il valore dell'etichetta **X** e il contenuto del registro **R5**. Il risultato di questa somma punta alla locazione di memoria il cui contenuto verrà poi caricato in **R2**.

6. Linguaggio Assembly e Direttive dell'Assemblatore

Struttura di un'Istruzione

Un'istruzione assembly è composta dai seguenti elementi:

- **Etichetta (facoltativa):** Un nome mnemonico associato all'indirizzo di quella specifica istruzione.
- **Operazione:** Il codice operativo che definisce l'azione da compiere.
- **Operandi:** Gli oggetti (registri, valori, indirizzi) su cui agisce l'operazione.
- **Commento (facoltativo):** Note aggiunte per chiarezza.

Direttive dell'Assemblatore

Le direttive sono istruzioni speciali rivolte non al processore, ma all'assemblatore stesso, per fornire informazioni su come gestire il programma:

- **DATA / WORD:** Indica all'assemblatore di riservare una parola di memoria e assegnargli un contenuto specifico.
- **EQU:** Utilizzata per associare un valore numerico a un'etichetta (spesso usata per definire le costanti).
- **ORIGIN:** Specifica il punto di partenza (l'indirizzo iniziale) del programma in memoria.
- **RESERVE:** Indica all'assemblatore di riservare una determinata quantità di spazio in memoria (espressa in byte).
- **END:** Segnala la fine fisica del codice del programma

Il Generatore di Segnali di Controllo nell'Architettura RISC

Il generatore di segnali di controllo può essere realizzato in due modalità: **cablata** o **microprogrammata**. La metodologia **cablata** è tipica delle architetture di tipo **RISC**.

In un'architettura RISC, il generatore è costituito da una rete di interconnessioni che produce segnali basandosi su diversi input:

1. **Segnali esterni:** Come quelli relativi alle operazioni di lettura e scrittura in memoria.
2. **Segnali di condizione:** Che derivano dallo stato dell'elaborazione.
3. **Contatore di passi:** Riceve il clock e permette al generatore di capire in quale fase dell'esecuzione ci si trovi.
4. **Decodificatore di istruzioni:** Riceve il contenuto dell'**IR** e comunica al generatore la tipologia esatta di istruzione in fase di esecuzione.

Un segnale di controllo può essere formalizzato tramite una **funzione logica**. Ad esempio, il segnale di **abilitazione del contatore** può essere espresso dall'operazione OR tra il segnale **MFC** e il negato del segnale **Wait MFC** (che indica se il processore è in attesa della memoria). Un altro esempio è il segnale di **abilitazione del PC**, derivante da funzioni logiche che coinvolgono i vari passi (T1) e segnali di condizione (come **TR** e **BR**).

L'Architettura CISC: Datapath e Bus

I processori **CISC** sono caratterizzati da istruzioni più complesse, che possono coinvolgere più operandi e occupare uno spazio superiore a una singola parola di memoria. Anche il **datapath** è strutturalmente differente rispetto al modello RISC: utilizza una rete di interconnessione che permette ai dati di viaggiare in modo **bidirezionale**.

In questa architettura non sono presenti i registri intermedi fissi, sostituiti da un **banco di registri temporanei**. La comunicazione tra le componenti avviene attraverso un **bus**. L'accesso a tale bus è regolato da una logica chiamata **Bus Driver**:

- Se l'input del Bus Driver è **0**, viene mantenuto lo stato attuale (l'ultimo componente che ha avuto accesso mantiene il controllo).
- Se l'input è **1**, una nuova componente richiede l'accesso; viene quindi abilitato un **flip-flop** che permette il passaggio dei dati sul bus.

Nell'architettura CISC, il sistema è spesso diviso in tre bus distinti: **Bus A**, **Bus B** e **Bus C**.

Esempi di Esecuzione in CISC

Per comprendere il funzionamento del sistema a tre bus, si possono analizzare due esempi:

1. **Istruzione ADD R5, R6:** Questa istruzione somma il contenuto di R5 ed R6, salvando il risultato in R5. Dopo il fetch e la decodifica, i contenuti di R5 e R6 vengono letti e caricati rispettivamente sui **Bus A** e **Bus B**. Questi sono inviati in input all'ALU; il risultato viene posto sul **Bus C** e infine inviato nuovamente al banco dei registri per sovrascrivere R5.

2. Istruzione AND X(R7), R9 (con indirizzamento indicizzato):

- **Fase iniziale:** Si effettua il fetch del PC tramite il Bus B, si legge l'istruzione in memoria e la si carica nell'**IR** tramite il Bus C.
- **Calcolo dell'indirizzo:** Si caricano i registri R7, R9 e il valore immediato (X). Il valore X viene caricato nel Bus C e salvato in un registro temporaneo (**Temp 1**).
- **Operazione ALU:** L'ALU somma il valore in Temp 1 con il contenuto di R7 (calcolo dell'indirizzo di memoria). Il risultato viene salvato in **Temp 2**.
- **Accesso Memoria:** Si legge il contenuto della memoria all'indirizzo calcolato in Temp 2 e lo si carica in Temp 1 (sovrascrivendo il vecchio valore X che non serve più).
- **Fase finale:** Si esegue l'operazione di **AND logico** tra Temp 1 e R9 tramite l'ALU, scrivendo il risultato finale in Temp 2.

Il Controllo Microprogrammato

La metodologia utilizzata in CISC si basa sul **controllo microprogrammato**. Per comprendere questo approccio, è necessario definire alcuni termini chiave:

- **Parola di controllo:** Rappresenta l'insieme dei segnali di controllo attivi in un determinato istante di tempo.
- **Microistruzione:** È l'insieme delle parole di controllo relative a un singolo passo di esecuzione.
- **Microutine:** L'insieme di microistruzioni necessarie per completare l'esecuzione di una specifica istruzione.
- **Microprogramma:** L'insieme completo di tutte le microutine di tutte le istruzioni supportate dal processore.

Questo sistema include un **generatore di indirizzi di microistruzioni** e un **Micro-PC** che punta alla microistruzione successiva da eseguire. Sebbene il controllo microprogrammato sia più semplice da realizzare a livello progettuale, risulta **più lento** rispetto a quello cablato; per questo motivo, l'architettura RISC predilige la soluzione cablata per massimizzare le prestazioni.

Pipelining: Esecuzione in Parallelo delle Istruzioni

1. Introduzione al Pipelining

Il **pipelining** non è altro che **l'esecuzione in parallelo di istruzioni**.

Necessità del Pipelining Senza l'utilizzo del pipelining, seguire le istruzioni sequenzialmente significherebbe che mediamente un'istruzione richiede circa **cinque cicli di clock**. Se in un programma ci fossero tre istruzioni, l'esecuzione totale richiederebbe 15 cicli di clock. Questa modalità fa perdere molto tempo, specialmente considerando che un programma contiene molte più di tre istruzioni. La pipeline aiuta a risolvere questo problema.

Concetto Operativo La pipeline permette l'esecuzione di più istruzioni contemporaneamente, anche se sono in stati diversi, operando come se fosse una **catena di montaggio**.

2. Modifiche al Datapath e Buffer Intermedi

Per implementare il pipelining, il datapath viene modificato I registri intermedi vengono sostituiti da **buffer di pipeline**(registri di pipeline), che separano uno stadio dal successivo.

Contenuto e Funzione dei Buffer Nei buffer intermedi non ci sono solo i dati che venivano salvati nei registri intermedi, ma sono salvati anche i **registri PC** (Program Counter) e **IR** (Instruction Register) e segnali di controllo.

Formalmente, si afferma che il **buffer di uno stato serve ad alimentare lo stato successivo**.

Di seguito, si descrive il flusso di alimentazione tra gli stadi della pipeline:

- **Buffer 1 (B1):** Alimenterà lo stato 2, ovvero lo stato di decodifica.
- **Buffer 2 (B2):** Alimenterà lo stato 3, che è lo stadio di elaborazione da parte dell'ALU (Arithmetic Logic Unit).
- **Buffer 3 (B3):** Alimenterà lo stadio della memoria.
- **Buffer 4 (B4):** Andrà ad alimentare lo stadio della scrittura nei registri (Write Back).

3. Esempio di Esecuzione Ideale con Tre Istruzioni

Si consideri l'esecuzione di tre istruzioni: ADD (Addizione), OR(OR logico) e SUB (Sottrazione), utilizzando il pipelining.

Ciclo di Clock	Stato 1: Fetch (B1)	Stato 2: Decode (B2)	Stato 3: Execute (B3)	Stato 4: Memory (B4)	Stato 5: Write Back (B5)
Ciclo 1	Prelievo Istr. ADD . PC punta ad ADD, IR contiene ADD.	Vuoto	Vuoto	Vuoto	Vuoto
Ciclo 2	Prelievo Istr. OR . PC punta all'OR, IR contiene OR.	Decodifica Istr. ADD . PC punta ad ADD.	Vuoto	Vuoto	Vuoto
Ciclo 3	Prelievo Istr. SUB . PC punta a SUB, IR è SUB.	Decodifica Istr. OR . PC punta all'OR.	Elaborazione/ALU di ADD (Somma). Risultato salvato in RZ nel buffer 3.	Vuoto	Vuoto
Ciclo 4	Prelievo Istr. 4 (Non esiste).	Decodifica Istr. SUB . PC punta a SUB.	Elaborazione/ALU di OR . Risultato salvato in RZ.	Accesso in Memoria di ADD . Risultato in Ry,	Vuoto

				alimentato da RZ dello stato precedente.	
Ciclo 5	Prelievo Istr. 5 (Non esiste).	Decodifica Istr. 4 (Non esiste).	Elaborazione/ALU di SUB.	Accesso in Memoria di OR.	Scrittura nei Registri di ADD.

4. Problematiche nel Pipelining (Hazard)

Il funzionamento ideale della pipeline presenta diverse problematiche che possono impedire l'esecuzione. La struttura della pipeline può non funzionare correttamente in alcune casistiche:

1. **Dipendenza di dato** (Data Hazard).
2. **Ritardi di memoria.**
3. **Istruzioni di salto** (Control Hazard).
4. **Limiti di risorse** (Resource Hazard).

4.1. Dipendenza di Dato (Data Hazard)

La dipendenza di dato si verifica quando **l'input di una seconda istruzione è l'output (risultato) della prima istruzione**. Se la seconda istruzione dipende dal risultato della prima, le due istruzioni non possono essere eseguite in contemporanea.

Esempio: AD R0, R1, R2 seguito da SUB R0, R0, R5. L'istruzione SUB usa R0, che è scritto da AD.

Soluzioni alla Dipendenza di Dato

1. **Aggiunta di Cicli di Attesa (NOP):** Si possono aggiungere dei cicli di attesa, introducendo istruzioni **NOP** (*No Operation*). Questo, tuttavia, va a svantaggio dell'efficienza della pipeline e ne limita le potenze.
2. **Inoltro degli Operandi (Forwarding):** Questo è un sistema per ovviare al ritardo. Consiste nel **passare direttamente il risultato della prima istruzione come input/operando per la seconda istruzione**.

L'inoltro degli operandi richiede una **modifica del datapath**. L'ALU, in questo caso, deve avere più multiplexer (MUX). Per l'input A, se il MUX è impostato a 0, il dato viene letto dal registro RA. Se il MUX è impostato a 1, il dato viene letto da **RZ**, ovvero dall'ultimo risultato dell'ALU. Un meccanismo analogo si applica all'input B.

4.2. Ritardi in Memoria (Cache Miss)

I ritardi in memoria derivano dalla **differenza di velocità tra processore e memoria**. È auspicabile che la lettura dalla memoria duri un solo ciclo di clock, ma questo accade solo se il dato si trova all'interno della **cache**. Se il dato non è scontato che si trovi in cache, si verifica un **cache miss**.

Soluzione L'unico modo per ovviare a un *cache miss* è introdurre **latenze**, ovvero nuove operazioni di attesa.

4.3. Ritardi nei Salti (Control Hazard)

L'efficienza della pipeline viene compromessa in presenza di **istruzioni di salto**. Il processore non riconosce che un'istruzione è di salto, e non conosce la sua destinazione, finché questa non viene **decodificata**. L'informazione sulla destinazione è disponibile solo allo **Step 3**.

Quando si arriva allo Step 3, la pipeline ha già prelevato **due istruzioni** che devono essere scartate, generando una **penalità di due salti**.

Soluzioni ai Ritardi nei Salti

1. **Identificazione Precoce della Destinazione:** Si modifica il datapath in modo che la lettura della destinazione del salto avvenga durante la decodifica dell'istruzione, cioè nello **Stadio 2**. In questo modo, si ha una penalità di **un solo salto**, riducendo la penalità complessiva.
2. **Predizione di Salto (Branch Prediction):** Per i **salti condizionati**, oltre a conoscere la destinazione nello Stadio 2, si fanno delle previsioni sul fatto che il salto venga effettuato o meno. Queste previsioni (predizioni) possono essere **statiche** o **dinamiche**.

Predizione dinamica (Automa a Due Stati)

Nella predizione dinamica a due stati si considera statisticamente che le possibilità di salto siano il **50% positive** (salto effettuato) e il **50% negative** (salto non effettuato).

Il processo è schematizzato tramite un **automa a due stati**:

- **Probabilmente Salta.**
- **Probabilmente Non Salta.**

Regole di Transizione (Predizione dinamica a due stati):

- Se si è nello stato *Probabilmente Salta* e il salto avviene, si rimane in *Probabilmente Salta*.
- Se si è in *Probabilmente Salta* e il salto non avviene, si passa allo stato *Probabilmente Non Salta*.
- Se si è in *Probabilmente Non Salta* e il salto non avviene, si rimane in *Probabilmente Non Salta*.
- Se si è in *Probabilmente Non Salta* e il salto avviene, si sposta allo stato *Probabilmente Salta*.

Predizione Dinamica (Automa a Quattro Stati)

La predizione dinamica a due stati è considerata meno veritiera della **predizione dinamica a quattro stati**. Questo automa include:

- **Molto Probabilmente Salta.**
- **Probabilmente Salta.**
- **Probabilmente Non Salta.**
- **Molto Probabilmente Non Salta.**

(Nota: Le regole di transizione sono più complesse, descritte nel dettaglio nei passaggi originali, ma il concetto chiave è l'aggiunta di stati intermedi per una previsione più robusta).

Meccanismo di Valutazione della Predizione Il processore decide se predire che il salto sarà effettuato o meno attraverso il **BTB (Branch Target Buffer)** o **Buffer di Destinazione Salto**.

Il BTB è una **tabella contenente**:

- L'indirizzo dell'istruzione di salto.
- L'indirizzo della destinazione di tale salto.
- **I bit di predizione.**

Questa tabella è considerata una **memoria veloce** e piccola, che non causa latenze per la sua lettura.

4.4. Limiti di Risorse (Resource Hazard)

I limiti di risorse si verificano quando **due stadi della pipeline tentano di accedere alla stessa risorsa hardware contemporaneamente**.

Esempio: Nello Stadio 1 (Fetch) si accede alla memoria per prelevare l'istruzione, e nello Stadio 4 (Memoria) si accede comunque alla memoria tramite istruzioni Load/Store. Se questi due stadi sono eseguiti contemporaneamente, si verifica uno **stallo**.

Soluzione Per evitare lo stallo causato dalla competizione per la memoria, si ricorre alla **separazione della cache per istruzioni e per dati**.

5. Valutazione delle Prestazioni

La valutazione delle prestazioni della pipeline può essere effettuata utilizzando specifiche formule.

Legenda simboli:

N: conteggio dinamico delle istruzioni.

S: numero medio di cicli di clock per completare l'istruzione.

R: frequenza di clock del processore.

Tempo di Esecuzione (T)

Il tempo di esecuzione è calcolato come segue:

$$T = \frac{N * S}{R}$$

Throughput (P)

Il Throughput (P) è il numero di istruzioni eseguite in un'unità di tempo.

$$P_{np} = \frac{R}{S}$$

Throughput con Pipeline Ideale (Senza Latenze) In una pipeline ideale, il Throughput è uguale al valore della **frequenza di clock**.

$$P_p = R$$

Throughput con Pipeline Reale (Con Latenze) Poiché nella vita reale non si ha una pipeline ideale senza latenze, la formula per il Throughput reale è data da

$$P_p = \frac{R}{1 + \delta}$$

(delta minuscolo) è il **valore di latenza di esecuzione** causato dalle varie problematiche (hazard).

Calcolo della Latenza Totale

$$\delta = \delta_{\text{stallo}} + \delta_{\text{penalita_salto}} + \delta_{\text{miss}}$$

Componente delta dovuta allo Stallo

La latenza dovuta allo stallo è data dalla moltiplicazione di tre fattori

$$\delta_{\text{stallo}} = \text{istruzioni_load} \times \text{istruzioni_dipendenti} \times \text{cicli_extra}$$

Componente delta dovuta alla Penalità di Salto

La latenza dovuta alla penalità di salto è data dalla moltiplicazione di tre fattori:

delta_Penalità di Salto = Percentuale di istruzioni di salto X Tasso di errore di predizione X Numero di cicli extra

Componente delta dovuto al Cache Miss

$$\delta_{\text{miss}} = (m_i + d \cdot m_d) \cdot p_m$$

p_m : il tempo di risposta della memoria RAM (pari a p_m cicli di clock).

m_i : frazione di istruzioni prelevate soggette a cache miss;

d : frazione di istruzioni Load o Store del conteggio dinamico;

m_d : frazione degli accessi a memoria soggetti a cache miss.

Il Funzionamento Superscalare del Processore

Il concetto di **funzionamento superscalare** si sviluppa nel contesto delle **pipeline**. Inizialmente, l'uso delle pipeline è stato introdotto per incrementare il **throughput**, ovvero il numero di istruzioni completate nell'unità di tempo, permettendo l'esecuzione contemporanea di più istruzioni. Il modello superscalare rappresenta un'evoluzione di questo concetto, mirata a migliorarne ulteriormente le caratteristiche prestazionali.

Per implementare un'architettura superscalare, è necessario apportare specifiche **modifiche all'hardware**. Una delle principali innovazioni riguarda l'**unità di prelievo (fetch unit)**, che deve essere in grado di prelevare più istruzioni contemporaneamente dal flusso del programma.

L'Organizzazione dell'Hardware e lo Smistamento

Una volta prelevate, le istruzioni alimentano una **coda**, la quale a sua volta fornisce i dati all'**unità di smistamento (dispatch unit)**. Il compito di quest'ultima è selezionare le istruzioni e indirizzarle verso le unità funzionali appropriate:

- **Unità aritmetico-logica (ALU):** dedicata alle operazioni matematiche e logiche.
- **Unità Load/Store:** dedicata alla gestione degli accessi alla memoria.

Queste unità lavorano in **parallelo**. Nonostante l'esecuzione avvenga su percorsi paralleli, i risultati confluiscono infine in un'unica **unità di scrittura**, responsabile di riportare i dati nei registri del processore.

Modifiche al Banco Registri

L'esecuzione parallela richiede un adeguamento strutturale del **banco registri (register file)**. Poiché l'ALU e l'unità di memoria operano simultaneamente, potrebbero richiedere l'accesso ai registri nello stesso momento, sia per leggere che per scrivere.

Se ipotizziamo il prelievo e l'esecuzione di due istruzioni contemporanee:

1. **In fase di lettura:** Ogni istruzione può richiedere fino a due operandi. Di conseguenza, il banco registri deve disporre di **quattro uscite** per permettere la lettura simultanea di quattro operandi.
2. **In fase di scrittura:** Se entrambe le unità producono un risultato, il banco registri deve disporre di **due ingressi** per scrivere i due risultati contemporaneamente.

In uno scenario ideale, le istruzioni vengono prelevate, decodificate e smistate a coppie, garantendo un accesso multiplo e fluido al banco registri.

Gestione delle Criticità: Salti e Dipendenze

Il funzionamento superscalare può incontrare ostacoli che ne riducono l'efficienza, come **dipendenze di dato, salti, cache miss ed eccezioni**.

- **Salti:** Per gestire le problematiche relative alle istruzioni di salto, si utilizzano le stesse metodologie della pipeline standard, ovvero il **buffer di destinazione di salto (BTB)** e le tecniche di **predizione di salto** tramite appositi bit di predizione.
- **Dipendenze di dato:** Per risolvere le dipendenze, si utilizzano le **stazioni di prenotazione (reservation stations)**. Si tratta di buffer di attesa dove gli operandi vengono parcheggiati. Un'unità può eseguire un'istruzione e comunicare il risultato solo dopo che **tutti gli operandi necessari sono arrivati**. In questo sistema, ogni unità comunica il proprio risultato a tutte le altre.

Gestione delle Eccezioni e Ordine di Esecuzione

Il parallelismo spinto dell'architettura superscalare può causare problemi in presenza di **eccezioni**. Se un'istruzione precedente nel codice genera un'eccezione, ma un'istruzione successiva (eseguita da un'unità diversa) viene completata prima, si

rischia di scrivere nel registro un risultato che teoricamente non dovrebbe esistere, poiché il flusso si sarebbe dovuto interrompere prima.

Un esempio tipico è la sequenza di istruzioni **Load** e **Sub**. Se la Load (che viene prima) genera un'eccezione, ma la Sub (che viene dopo) viene calcolata rapidamente dall'ALU, il risultato della Sub non deve essere scritto nei registri.

Unità di Commitment e Buffer di Riordino

Per garantire la coerenza, si impone che l'**ordine finale di impegno (commit o commitment)** delle istruzioni sia identico all'ordine originale del codice. Vengono introdotti due elementi chiave:

1. **Registri temporanei:** Risultati intermedi vengono salvati qui invece che direttamente nel banco registri finale. Se avviene un'eccezione, i risultati delle istruzioni successive vengono scartati e non "serviti".
2. **Buffer di riordino (Reorder Buffer):** Garantisce che l'ordine dell'unità di commitment coincida con l'ordine di smistamento.

La regola fondamentale è che, sebbene tra la fase di smistamento e quella di impegno le istruzioni possano essere gestite e ordinate liberamente, **l'ordine di ingresso (dispatch) e quello di uscita (commit) devono essere identici** a quelli del flusso di istruzioni originale.

Lo Smistamento e il Rischio di Deadlock

L'unità di smistamento ha anche il compito di verificare la **disponibilità delle risorse** prima di abilitare un'istruzione. Deve controllare che ci sia spazio nelle **stazioni di prenotazione**, che siano disponibili i **registri temporanei** e che vi sia una locazione libera nel **buffer di riordino**.

Sebbene sia possibile smistare le istruzioni fuori ordine per ottimizzare i tempi, è necessario evitare il **deadlock (stallo)**. Il deadlock si verifica quando due unità dipendono l'una dal risultato dell'altra e rimangono in attesa infinita, bloccando il sistema. Per prevenire questa situazione, in certi casi si evita di emettere istruzioni fuori ordine.

Architetture Moderne e Differenze tra RISC e CISC

I processori superscalari moderni sono macchine estremamente complesse che tipicamente includono:

- Due unità aritmetiche per operazioni con **interi**.

- Un'unità per operazioni in **virgola mobile (FPU)**, dotata di un proprio banco registri.
- Un'unità **vettoriale**, capace di svolgere da due a otto operazioni in parallelo, anch'essa con un banco registri dedicato.
- Una singola unità **Load/Store**, che può supportare il prelievo di quattro o più istruzioni contemporaneamente per portarle nella coda.

Complessità nei Processori CISC

Nei processori con architettura **CISC**, l'implementazione della pipeline e del sistema superscalare è molto più complessa rispetto ai sistemi RISC. Le difficoltà principali includono:

- **Dimensione variabile delle istruzioni**, che possono occupare più di una parola di memoria.
- **Localizzazione incognita degli operandi** e modi di indirizzamento complessi.
- **Durata imprevedibile del prelievo** delle istruzioni, che complica notevolmente sia la fase di decodifica che quella di smistamento.

Rappresentazione delle Variabili Binarie e Funzionamento dei Transistor

1. La Rappresentazione delle Variabili Binarie nei Circuiti Logici

Nei circuiti logici, per stabilire se l'output debba corrispondere al valore **0** o al valore **1**, si ricorre al confronto della tensione del circuito con un valore predefinito. Questo valore è definito **soglia di separazione**.

Il principio di funzionamento è il seguente:

- Se la tensione del circuito supera il valore di soglia, il risultato (output) è **1**.
- Altrimenti (se non supera la soglia), il risultato (output) è **0**.

La Banda Vietata (Range Non Considerato)

I circuiti logici non prendono in considerazione i valori che sono numericamente vicini alla soglia di separazione. Questo intervallo di valori viene ignorato in quanto potrebbero derivare da un errore o da un rumore (disturbo) all'interno del circuito. Tali valori, vicini alla soglia, non rappresenterebbero quindi un dato o un valore effettivo. Questo specifico intervallo di tensioni non considerato è definito **banda vietata**.

2. I Transistor MOS come Interruttori Elettronici

Il transistor più diffuso e comune è il **MOS**.

Un esempio tipico di configurazione MOS prevede i seguenti valori di riferimento per la tensione:

- La tensione di alimentazione (V_{cc}) è **5V**.
- La massa è **0V**.
- La soglia di separazione (o soglia operativa) è la metà del V_{cc} , ovvero **2,5V**.

Struttura del Transistor MOS

I transistor MOS sono costituiti da **tre elementi principali**:

1. La **sorgente**.
2. Il **pozzo** (spesso chiamato Drain).
3. La **porta** o **base** (spesso chiamata Gate). Decide se il canale è aperto o chiuso.

3. Tipologie di Transistor MOS: nMOS e pMOS

Esistono due versioni fondamentali di transistor MOS: l'**nMOS** e il **pMOS**.

Il transistor MOS può essere visto come un interruttore controllato dal gate: nel caso dell'**nMOS** l'interruttore collega l'uscita a massa quando è chiuso, mentre nel **pMOS** collega l'uscita a V_{cc} quando è chiuso.

Cosa succede quando è aperto? Il transistor non forza alcun valore sull'uscita. La configurazione CMOS, con nMOS e pMOS complementari, che garantisce che l'uscita assuma sempre un livello logico definito.

4. Implementazione delle Porte Logiche con Transistor nMOS

I circuiti logici, incluse le porte, sono tipicamente costituiti da elementi fondamentali: una **sorgente**, una **porta** (o gate), un **pozzo** (o drain) e un pozzo di alimentazione che è collegato tramite una **resistenza**.

Il Circuito NOT

I circuiti NOT sono realizzati utilizzando i transistor **nMOS**. Il circuito NOT ha la funzione di negare l'ingresso:

- Se l'interruttore è **chiuso**, l'uscita (output) sarà **zero**.
- Se l'interruttore è **aperto**, l'uscita sarà **uno**.

Il Circuito NOR (NOT OR)

Per realizzare la porta logica NOR, si parte dalla configurazione di base e si utilizzano **due interruttori in parallelo**. La porta NOR avrà come uscita **uno** soltanto quando **entrambi** gli interruttori sono **aperti**.

Il Circuito NAND (NOT AND)

Il circuito NAND, noto anche come AND negato, si costituisce partendo dalla porta logica di base, ma si utilizza l'implementazione di **due interruttori in serie**, a differenza del NOR che li usa in parallelo. In questo caso, l'uscita sarà **zero** solo quando **entrambi** gli interruttori sono **chiusi**.

Realizzazione di AND e OR

Le porte logiche AND e OR possono essere costruite a partire dalle loro versioni negate:

- Per ottenere la porta **AND**, è sufficiente collegare un circuito **NOT in serie** a una porta NAND.
- Per ottenere la porta **OR**, è necessario collegare un circuito **NOT in serie** a una porta NOR.

5. La Tecnologia CMOS e i Vantaggi sulla Tecnologia nMOS

Il principale problema associato all'uso dei soli transistor nMOS è l'elevato **dispendio di tensione** e il **rilevato consumo di energia**.

Per risolvere questa problematica, si ricorre alla **tecnologia CMOS (Complementary MOS)**, che è complementare a MOS.

Struttura e Benefici del CMOS

La tecnologia CMOS consiste nel collegare un transistor **nMOS in serie a un transistor pMOS**.

Questa configurazione risulta anche più **stabile**. I vantaggi del CMOS includono:

- Un **minor dispendio di energia**.
- Una **potenza elettrica dissipata** che è proporzionale alla **frequenza di commutazione**.
- Il fatto che i transistor CMOS siano più **ridotti** (in dimensione), il che permette di massimizzare la **frequenza di commutazione**.

6. Ritardi Operativi, Fan-in e Fan-out

I ritardi che possono verificarsi in un circuito logico sono generalmente causati da due fattori principali: il **ritardo nella propagazione del segnale** e il **tempo impiegato dai transistor per transitare di livello**.

Fan-in e Fan-out

In una porta logica:

- Il **Fan-in** rappresenta il numero degli **ingressi** (input).
- Il **Fan-out** rappresenta il numero delle **uscite** (output).

Avere un numero elevato di Fan-in e Fan-out in una porta logica incide negativamente sia sui **ritardi di propagazione** che sui **margini di rumore**. Per questo motivo, generalmente, il numero di Fan-in e Fan-out per porta **non supera i 10**.

7. Circuiti Integrati Complessi

Il Decoder (Decodificatore)

Il decoder è un circuito integrato che ha la funzione di **decodificare i dati in binario**. In virtù di questa funzione, se il decoder ha N ingressi, avrà 2^N uscite.

Il Multiplexer (MUX)

Il multiplexer è un altro circuito integrato importante, utile anche nei processori CISC e RISC. Il MUX si distingue perché, oltre ad avere **diversi input** e **un solo output**, include in ingresso anche uno o più **selettori**.

Il multiplexer possiede $2N$ **ingressi**, dove N è il numero dei selettori, e ha **una singola uscita**. Il suo scopo è quello di **scegliere** uno dei dati che arrivano in input, guidato dal segnale ricevuto dal **selettore**.

8. Reti Sequenziali (Bistabili)

Fino a questo punto, l'analisi si è concentrata sui circuiti in cui l'uscita dipende esclusivamente dall'input corrente o dal selettore.

Tuttavia, in molte applicazioni (come gli stati di una cassaforte), è necessario considerare anche gli **stati precedenti** o gli **input precedenti**. I circuiti che tengono conto di questa dipendenza sono chiamati **reti sequenziali**.

Le reti sequenziali sono anche denominate **bistabili**, e possono essere di tre tipi principali: **asincroni**, **sincroni** e di **tipo D**.

9. Tipologie di Bistabili

Bistabile Asincrono (Flip-Flop RS)

Il bistabile asincrono è costituito da una coppia di porte logiche **NOR**, collegate in modo tale che **uno dei loro ingressi sia connesso all'uscita dell'altra**.

Questa configurazione fa sì che, generalmente, l'uscita di una porta sia uguale al **negato dell'uscita dell'altra**. Il bistabile asincrono ha due ingressi principali, che vengono chiamati **R** (Reset) e **S** (Set).

La regola della negazione dell'uscita non viene rispettata in due casi specifici, cioè quando gli ingressi **R e S sono entrambi a zero** oppure **entrambi a uno**.

- Il caso in cui **R e S sono entrambi a uno non viene considerato** per evitare ambiguità.
- Il caso in cui **R e S sono entrambi a zero** è considerato come **assenza di segnale** e in tale situazione non si considera lo stato corrente, ma lo **stato precedente**.

Bistabile Sincrono

Il bistabile sincrono aggiunge un input di controllo, il **CLK** (Clock), oltre agli ingressi S e R. È tramite il clock che viene indicato lo stato che si sta analizzando.

- Quando il clock (CLK) è a **uno**, il bistabile si comporta come un bistabile asincrono.
- Quando il clock è a **zero**, lo stato **non cambia**.

Bistabile di Tipo D (Data)

Nel bistabile di tipo D, gli ingressi S e R vengono combinati in un unico ingresso chiamato **D** (dato). In questa configurazione, l'ingresso **R è uguale al negato di S**, e **S è uguale a D**.

- Quando il clock è **uno**, l'uscita è uguale a **D**.

- Se il clock è **zero**, lo stato **non cambia**

Sistemi di Memoria: Componenti e Valutazione

I **sistemi di memoria** comprendono tutte le componenti di un calcolatore che permettono l'immagazzinamento dei dati. Questi si suddividono principalmente in:

- **Memoria principale:** che include RAM, ROM e memoria Cache.
- **Memoria secondaria:** ovvero le memorie di massa.

Per valutare l'efficacia di una memoria, è necessario considerare diversi parametri fondamentali:

1. **Velocità (Tempo di accesso):** indica la rapidità con cui si accede ai dati. La velocità è inversamente proporzionale al tempo di accesso.
2. **Latenza (Tempo di ciclo di memoria):** definibile come la differenza di tempo minima che intercorre tra un'operazione in memoria e quella successiva.
3. **Costo:** il prezzo della memoria per unità di memorizzazione.
4. **Larghezza di banda:** la quantità di dati che le memorie possono fornire contemporaneamente.
5. **Capacità:** il volume totale di dati che possono essere immagazzinati.

La Memoria RAM: Tipologie e Funzionamento

La **RAM** è una memoria **volatile**, il che significa che necessita di alimentazione elettrica costante per mantenere i dati conservati. Dal punto di vista strutturale, può essere vista come una **matrice di celle** organizzata in righe da 8 bit; la linea orizzontale (linea di riga) identifica una **parola** (composta da 32 bit), mentre le linee verticali servono per le operazioni di lettura e scrittura. Tali operazioni sono gestite da **segnali di controllo**: se il segnale *read* è impostato a 1 si sta leggendo, se è a 0 si sta scrivendo.

Complessivamente, si possono individuare circa 17 collegamenti: quattro linee di indirizzo, otto linee di dati, due linee di controllo e due di alimentazione.

Esistono due tipologie principali di RAM:

RAM Statica (SRAM)

La **SRAM** è considerata superiore in termini di efficienza ma ha un costo maggiore. Le sue caratteristiche principali sono:

- **Mantenimento dati:** non necessita di operazioni di refresh; finché è alimentata, mantiene i dati.
- **Struttura interna:** ogni cella è generalmente costituita da **sei transistor**. Semplificando lo schema a due transistor (T1 e T2), questi sono collegati tramite un circuito di porte NOT in modo che l'uno riceva il segnale negato dell'altro.

- **Stati di lavoro:** il circuito lavora in **saturazione** (lettura) se il valore della linea di parola è 1, o in **interdizione** (scrittura) se il valore è 0.

RAM Dinamica (DRAM)

La **DRAM** è più economica ma presenta degli svantaggi strutturali:

- **Necessità di Refresh:** anche se alimentata, può perdere i dati poiché è costituita da un transistor e un **condensatore**.
- **Funzionamento:** quando la linea di parola è a 1 (saturazione), il bit è contenuto nel condensatore; quando si passa in interdizione (valore 0), il condensatore viene scollegato e tende a scaricarsi gradualmente, causando la perdita del dato se non viene "rinfrescato" (ovvero riletto e riscritto) periodicamente.

Organizzazione dei Collegamenti e Multiplexing

Per individuare una cella in memoria sono necessarie le coordinate di riga e colonna. Tuttavia, fornire contemporaneamente entrambi gli indirizzi richiederebbe un numero eccessivo di collegamenti hardware.

Per ovviare a questo problema, si utilizza il **multiplexing degli indirizzi**: il processore invia l'indirizzo di riga e quello di colonna in istanti temporali diversi utilizzando lo stesso collegamento. Questo processo è gestito da due decodificatori:

- **RAS (Row Address Strobe):** per la riga.
- **CAS (Column Address Strobe):** per la colonna.

Inoltre, la memoria può essere organizzata in **moduli** (gruppi di chip). In questo caso, il campo indirizzi contiene dei bit per identificare il modulo specifico, chiamati **Chip Select**, e dei bit per identificare il dato all'interno del modulo stesso.

Gerarchia della Memoria e Principi di Località

Le memorie sono organizzate gerarchicamente per bilanciare velocità e costi.

- **In ordine di velocità e costo decrescente (dalla più veloce alla più lenta):** Registri, Cache L1, Cache L2, Memoria Principale, Memoria Secondaria.
- **In ordine di capacità crescente (dalla più piccola alla più grande):** Registri, Cache L1, Cache L2, Memoria Principale, Memoria Secondaria.

La gestione dei dati tra i vari livelli, specialmente verso la Cache, si basa su due criteri:

1. **Principio di Località Spaziale:** se viene utilizzato un dato, è molto probabile che verranno utilizzati a breve anche quelli adiacenti (contigui).
2. **Principio di Località Temporale:** se un dato è stato usato recentemente, è probabile che verrà richiesto di nuovo a breve.

Questi principi sono evidenti nell'uso degli **array (vettori)**: quando si accede a un elemento, serve spesso scorrere i successivi (spazialità) e riutilizzare il vettore per più operazioni come l'aggiornamento o l'inizializzazione (temporalità).

Impatto sulle Prestazioni:

Se l' Hit Rate è alto il Tempo medio di accesso drasticamente ridotto. La cache "nasconde" la latenza della RAM

Esempio: se cache hit = 1 ciclo, RAM = 100 cicli, hit rate 90% → Tempo medio = $0.9 \times 1 + 0.1 \times 100 = 10.9$ cicli (vs 100 senza cache)

Formula delle Prestazioni (CPU Time)

$$\text{Tempo_CPU} = N_{\text{istruzioni}} \times \text{CPI} \times T_{\text{clock}}$$

Dove:

- $N_{\text{istruzioni}}$ = Numero totale di istruzioni del programma

(dipende dall'algoritmo e dal compilatore)

- CPI (Cycles Per Instruction) = Numero medio di cicli per istruzione

(dipende dall'ISA, dalla pipeline, dagli hazard)

- T_{clock} = Tempo di un ciclo di clock = $1/\text{Frequenza}$

(dipende dalla tecnologia dei transistor, dall'organizzazione hardware)

Per migliorare le prestazioni si può agire su:

1. Ridurre $N_{\text{istruzioni}}$ → ISA più efficiente, compilatore migliore
2. Ridurre CPI → Pipeline, superscalare, riduzione hazard
3. Ridurre T_{clock} → Tecnologia più veloce (transistor più piccoli)

Nota: Frequenza alta ≠ prestazioni alte se CPI peggiora!

La Memoria Cache: Funzionamento e Gestione

La **Cache** è una memoria estremamente veloce ma di piccole dimensioni, situata tra il processore e la memoria principale per velocizzare la lettura e scrittura dei dati.

Cash Hit e Cash Miss

- **Cache Hit:** si verifica quando il processore trova il dato necessario direttamente nella Cache.
 - *Lettura:* avviene immediatamente.
 - *Scrittura:* può essere **immediata (Write-Through)**, scrivendo subito in memoria principale, o **differita (Write-Back)**, dove la scrittura in RAM avviene solo poco prima che la posizione in Cache venga svuotata.
- **Cache Miss:** si verifica quando il dato non è presente in Cache e deve essere recuperato dalla memoria principale, comportando una perdita di tempo.
 - *Lettura:* si può caricare l'intero blocco di dati (**Read-Back**) o solo la parola specifica richiesta (**Load-Through**).
 - *Scrittura:* con il **Write-Back** si carica prima il blocco in Cache e poi si aggiorna la parola; con il **Write-Through** si aggiorna direttamente la parola in memoria.

Indirizzamento della Cache

Esistono tre metodi per associare i blocchi della RAM alle posizioni in Cache:

1. **Indirizzamento Diretto:** ogni blocco di memoria corrisponde a una posizione fissa in Cache.
 - a. *Vantaggi:* ricerca molto rapida.
 - b. *Svantaggi:* scarsa flessibilità; un blocco occuperà sempre la stessa posizione indipendentemente dal momento.
 - c. *Campi:* **Blocco** (posizione in cache), **Spiazzamento** (parola nel blocco) ed **Etichetta** (per il confronto tra cache e memoria).
2. **Indirizzamento Associativo:** un blocco può occupare qualsiasi posizione in Cache.
 - a. *Vantaggi:* elimina i vincoli di posizione, migliorando lo sfruttamento della capacità.
 - b. *Svantaggi:* ricerca e confronto delle etichette molto più complessi.
 - c. *Campi:* Solo **Etichetta** e **Spiazzamento**.
3. **Indirizzamento Associativo a Gruppi:** una via di mezzo tra i precedenti. La memoria è divisa in gruppi; un blocco ha una posizione fissa all'interno di un gruppo, ma può far parte di gruppi diversi in momenti diversi.
 - a. *Campi:* **Gruppo**, **Spiazzamento** ed **Etichetta**.

Aggiornamento e Sostituzione dei Dati

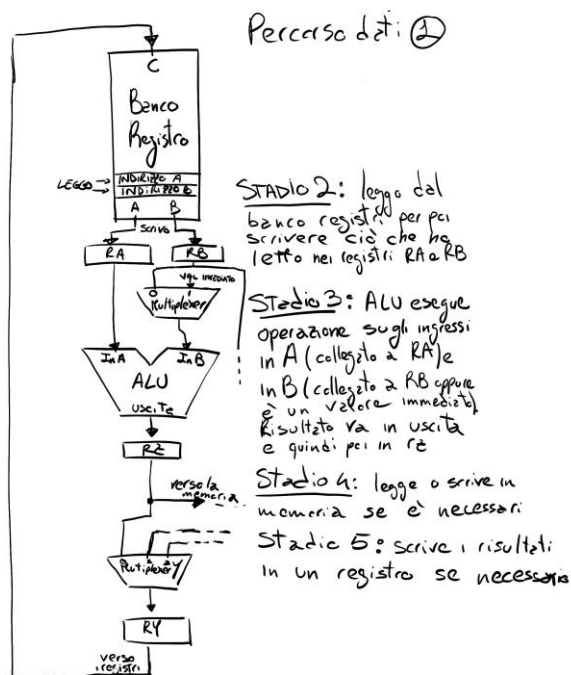
La coerenza dei dati può essere compromessa dal **DMA (Direct Memory Access)**, che può aggiornare la memoria principale indipendentemente dal processore. Se il DMA scrive in memoria, i dati corrispondenti in Cache devono essere **invalidati**; se il DMA legge dalla memoria, la Cache deve essere svuotata.

Per decidere quali dati rimuovere quando la Cache è piena, si usa l'algoritmo **LRU (Least Recently Used)**. Questo sistema associa a ogni posizione un contatore (o semaforo):

- Il contatore aumenta quanto più a lungo la posizione resta inutilizzata.
- Il contatore si azzerà ogni volta che la posizione viene letta o scritta.
- Quando serve spazio, viene liberata la posizione con il contatore più alto, applicando così il **principio di località temporale**.

Allegati immagini:

Stadi processore risc (datapath)



Percorso dati

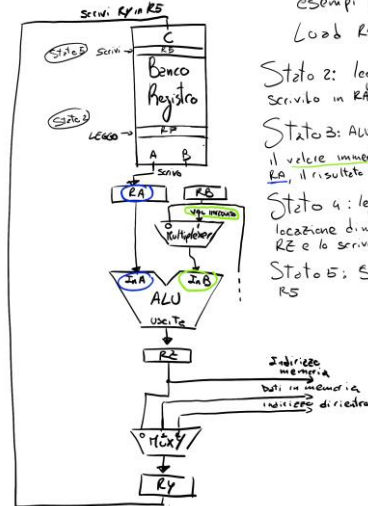
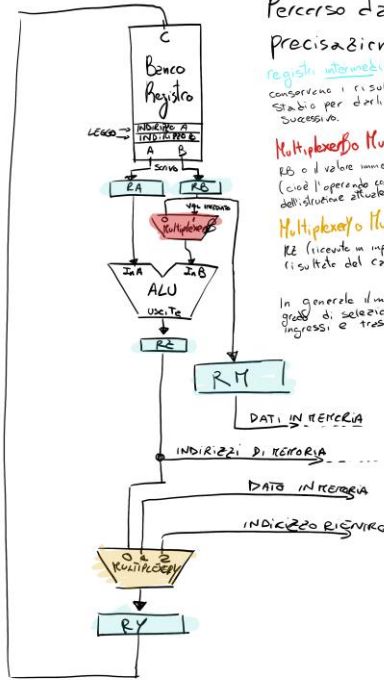
Precisazione Componenti:

registri intermedi (RA, RB, RE, RY):
conservano i risultati prodotti da uno stadio per darli in input allo stadio successivo.

Multiplexor MuxB: invia il contenuto di RB o il valore immediato contenuto in IR (cioè l'operando con il valore immediato dell'istruzione attuale).

Multiplexor MuxY: prende il registro RE (ricevuto in input) trasferisce il risultato del calcolo in RY.

In generale il multiplexor è in grado di selezionare uno dei tanti ingressi e trasferirlo in uscita.



Esempi pratici

Load R5, X[R5]

Stato 2: leggi R5 dai registri e scrivilo in RA

Stato 3: ALU esegue somma tra il valore immediato e il registro RA, il risultato viene scritto in RE

Stato 4: leggi il dato nella locazione di memoria dell'indirizzo RE e lo scrivi in RY

Stato 5: Scrivo RY nel registro R5

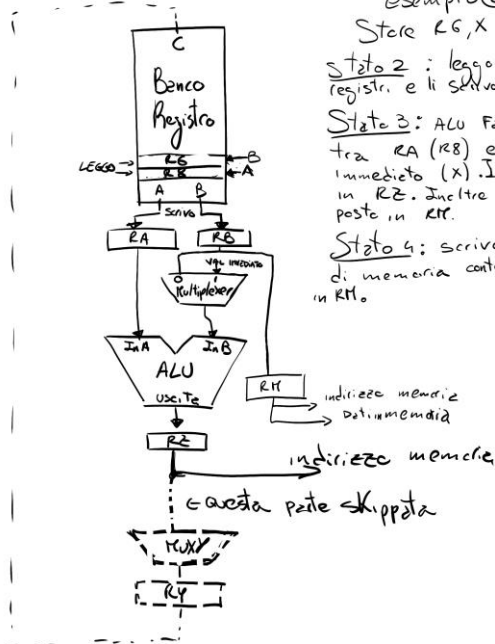
Esempio (2)

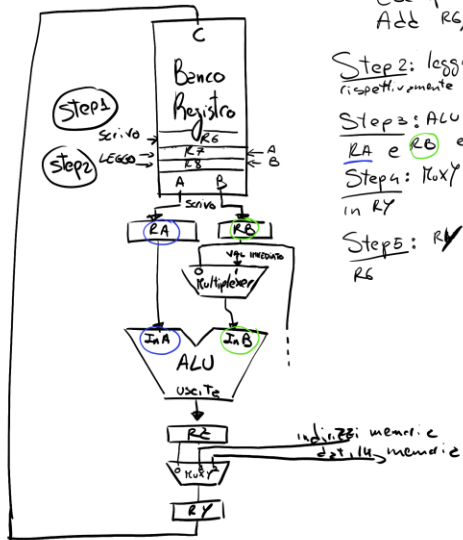
Store R6, X[R5]

Stato 2: leggo R6 e R5 dai registri e li scrivo in RA e RB

Stato 3: ALU fa la somma tra RA (R5) e valore immediato (X). Il risultato va in RE. Inoltre RB viene posto in RM.

Stato 4: scrivo l'indirizzo di memoria contenuto in RE nel RM.





Esempio ③
`ADD R6, R8, R7`

Step 2: leggo R_8 e R_7 e scrivo
rispettivamente su R_A e R_B

Step 3: ALU calcola la somma tra
 R_A e R_B e la scrive su R_C

Step 4: MUX seleziona R_C e lo trasferisce
in R_Y

Step 5: R_Y viene scritto nel registro
 R_6